

Resumen técnico — CodelabsBillingCloud

Table of Contents

1. Contexto y metadatos	2
2. Propósito funcional	2
3. Arquitectura	3
4. Interfaces y contratos	4
5. Flujos técnicos	5
5.1 Emisión de un comprobante electrónico (factura) hasta autorización SRI	5
5.2 Confirmación de pago de suscripción (dLocal Go)	5
5.3 Onboarding self-service con OTP	6
6. Seguridad y contexto de ejecución	6
7. Persistencia	7
8. Catálogo de implementación	8
9. Operación y configuración	8
10. Riesgos y deuda	9
Anexo — Evidencia transversal	11
Patrón de decorador Lambda (contrato compartido de todos los handlers HTTP)	11
Orden de decoradores en endpoints mutables	12
Idempotencia HTTP (X-Idempotency-Key)	12
Outbox pattern (eventos de dominio)	13
Cliente HTTP del frontend — refresco de token y envelope uniforme	13
Sesión y almacenamiento de tokens (frontend)	13
Convención de nombres de recursos AWS vs. marca del producto	13
Anexo — Arquitectura y flujos técnicos (detalle ampliado)	14
Flujo A — Emisión de factura hasta autorización SRI (ampliación de la sección 5.1)	14
Flujo B — Pago de suscripción con dLocal Go SmartFields (ampliación de la sección 5.2)	15
Flujo C — Onboarding self-service con OTP (ampliación de la sección 5.3)	16
Flujo D — Ciclo de vida del certificado digital (resumen)	16
Anexo — Backend (backend/)	17
Capa compartida (backend/shared/)	17
Capa transversal de Lambda (backend/lambda/_base/)	18
Dominios HTTP — estructura verificada	18
Workers (backend/lambda/workers/) — disparadores verificados en infra/stacks/api_stack.py	19
Tests (backend/tests/)	20
Anexo — Frontend (frontend/)	20
Enrutamiento (frontend/app/ , Expo Router — convención de archivos)	21

Features (<code>frontend/features/<dominio>/</code>) — paralelismo con dominios backend	21
Infraestructura transversal (<code>frontend/lib/</code>)	22
Autenticación y sesión (frontend)	22
Configuración de entorno (<code>frontend/constants/config.ts</code>)	22
Tests (frontend)	22

1. Contexto y metadatos

Repositorio: `CodelabsBillingCloud` (`wali-billing`). Fuente: rama local `develop`, último commit (excepción registrada: remoto inalcanzable al momento de esta instalación, ver `AGENTS.md`). SHA: `09a4a3b2aa87f139c4836733180edf87371c14b3` (commit `chore: retirar CLAUDE.md, context/ y docs/ (gestor de memoria y documentación 1.1)`). No incluye los ~82 archivos con cambios reales sin commitear presentes en el checkout local (verificado con `git status --short` antes de generar esta documentación: 82 entradas).

Tecnología:

- Backend: Python 3.12 sobre AWS Lambda (arquitectura ARM64/Graviton2), organizado en `backend/lambda/` por dominio funcional, con capa compartida en `backend/shared/`. Lint/format con `ruff`, seguridad estática con `bandit`, auditoría de dependencias con `pip-audit`, tests con `unittest` + `coverage` (umbral `fail_under = 70`, ver `backend/pyproject.toml`).
- Frontend: Expo / React Native (`frontend/`, main: `expo-router/entry`, Expo SDK ~56, React 19.2.3, React Native 0.85.3), enrutado con `expo-router` (convención de carpetas en `frontend/app/`), estado con `zustand`, validación con `zod`, formularios con `react-hook-form`, tests con `vitest`.
- Infraestructura: AWS CDK en Python (`infra/`), un stack por responsabilidad (`DatabaseStack`, `AuthStack`, `QueuesStack`, `StorageStack`, `ApiStack`, `CertificateStack`, `FrontendStack`).

Estructura inmediata del repositorio (raíz de `develop`): `Makefile`, `backend/` (`lambda/`, `shared/`, `tests/`, `migrations/`), `frontend/` (`app/`, `features/`, `lib/`, `components/`, `constants/`, `assets/`, `scripts/`, `public/`), `infra/` (`stacks/`, `config/`), `postman/`, `scripts/`, `.github/` (`workflows pr-checks.yml`, `deploy-dev.yml`, `deploy-staging.yml`, `deploy-prod.yml`).

Dependencias runtime principales — backend (`backend/requirements.txt`): `boto3/botocore` 1.43.24, `pydantic` 2.13.4, `cryptography` 48.0.1, `lxml` 6.1.1, `reportlab` 4.5.1 (generación de RIDE en PDF), `python-dateutil`. CDK (`infra/requirements.txt`): `aws-cdk-lib` 2.258.0, `constructs` 10.6.0, `jsii` 1.134.0, `typeguard` 2.13.3 (fijado: "v3+ es incompatible con CDK Python"). Frontend (`frontend/package.json`): `expo`, `expo-router`, `expo-secure-store`, `react-hook-form`, `zod`, `zustand`.

2. Propósito funcional

CodelabsBillingCloud ("Wali") es una plataforma SaaS multitenant de facturación electrónica para Ecuador (integración con el SRI — Servicio de Rentas Internas) con cobro de suscripción propio vía dLocal Go. El alcance real verificado en `backend/lambda/` cubre los siguientes dominios (13 carpetas bajo `lambda/`, incluyendo `_base` y `workers`):

- `auth` — login/refresh/logout/challenge y recuperación de contraseña contra Amazon Cognito

(USER_SRP_AUTH).

- **onboarding** — registro self-service de un tenant nuevo vía OTP por email, con bifurcación a un "lead empresarial" (**ConfirmOnboardingOtpUseCase**) cuando el caso no es autoservicio.
- **subscriptions** — cobro de la suscripción del tenant a la propia plataforma vía dLocal Go SmartFields (crear pago, confirmar con tarjeta, consultar estado, reembolsar, webhook HMAC-firmado).
- **tenants** — CRUD de tenants, cambio de plan, activación/renovación/reintento de suscripción, panel superadmin.
- **certificates** — carga y validación del certificado digital **.p12** de cada tenant (obligatorio para firmar comprobantes ante el SRI).
- **sequences** — establecimientos y puntos de emisión SRI, contador atómico de secuencial por punto de emisión.
- **products** — catálogo de productos/servicios facturables del tenant, con campañas de descuento.
- **clients** — catálogo de clientes del tenant (compradores de sus facturas).
- **documents** — emisión de comprobantes electrónicos (factura tipo **01**, nota de crédito tipo **04**), listado, resumen, descarga de RIDE/XML, anulación (legacy) y reintento.
- **invoice_processor** — no expone HTTP: procesa mensajes SQS para firmar XAdES-BES y enviar/consultar autorización ante el SRI.
- **workers/*** — diez procesos asíncronos (colas SQS o EventBridge) descritos en la sección 5 y en **modules/backend.adoc**.

No se encontró evidencia de módulos de contabilidad, reportes financieros avanzados o multi-país: el dominio SRI es específico de Ecuador (ver **xml_builder.py**, **sri_client.py**, claves de acceso de 49 dígitos). El alcance está limitado a lo verificado en el código de **develop**; no se infiere nada del nombre del repositorio.

3. Arquitectura

Cada dominio bajo **backend/lambda/<dominio>** sigue, con variaciones según su complejidad, una capa de arquitectura limpia verificada directamente en el árbol de archivos:

- **domain/** — comandos, errores, entidades, value objects y **puertos** (interfaces **i_*.py**) sin dependencias de AWS.
- **use_cases/** — orquestación de reglas de negocio contra los puertos del dominio.
- **infra/** — implementaciones concretas de los puertos (repositorios DynamoDB, cliente Cognito, cliente dLocal, cliente SOAP del SRI, storage S3).
- **handler.py** — adaptador AWS Lambda (API Gateway HTTP o SQS) que instancia infraestructura en cold start y enruta por método+path (o por **type** del mensaje en colas).
- **schemas.py** — validación de payloads con **pydantic**.

No todos los dominios tienen las cuatro subcapas: **certificates** no tiene **domain//infra/** propios (reutiliza **shared/certificates/** y el repositorio de **tenants**); **invoice_processor** no tiene **schemas.py**

(no es HTTP) y añade módulos de dominio específicos en su raíz (`xml_builder.py`, `signing.py`, `sri_client.py`, `ride_builder.py`, `sri_error_classifier.py`, `sri_error_mapper.py`, `events.py`, `ports.py`). Detalle completo por dominio en `modules/backend.adoc`.

Capa compartida (`backend/shared/`): `db/base_repository.py` (repositorio DynamoDB base con aislamiento multitenant, soft delete, locking optimista y auditoría automática — ver cita en sección 7), `db/client.py`, `db/transactions.py`, `db/paginator.py`, `db/counts.py`, `db/locks.py`; `audit/writer.py`; `certificates/` (validación y almacenamiento de .p12); `secrets/client.py` (AWS Secrets Manager); `domain/base_entity.py`, `domain/events/` (outbox pattern), `domain/value_objects/`; `logger.py` (logging estructurado con contexto de invocación); `errors.py` (jerarquía `AppError`); `config.py`.

Capa transversal Lambda (`backend/lambda/_base/`): `handler.py` (decoradores `lambda_handler/public_lambda_handler`), `sqs_handler.py` (equivalente para colas), `permissions.py` (`require_superuser/require_role`), `idempotency.py`, `parser.py` (`Request`, `parse`), `response.py` (`ApiResponse`), `query_params.py`.

Infraestructura (`infra/stacks/`, AWS CDK Python): `database_stack.py` (13 tablas DynamoDB, ver sección 7), `auth_stack.py` (Cognito User Pool único con roles `superadmin|owner|admin|viewer` vía `custom:role`), `queues_stack.py` (colas SQS con DLQ), `storage_stack.py` (S3 con Object Lock WORM para XML/RIDE legal), `api_stack.py` (22 funciones Lambda + HTTP API Gateway v2 con autorizador JWT nativo contra Cognito — el archivo más extenso de infra, ~1200 líneas), `certificate_stack.py` / `frontend_stack.py` (CloudFront + Route53, cross-region para el certificado ACM de CloudFront en `us-east-1`).

Frontend (`frontend/`): Expo Router con carpetas de grupo (`public/`), (`auth/`), (`app/tenant/`), (`app/superadmin/`) — separación de layout por rol. Lógica de negocio en `frontend/features/<dominio>/` (paralelo a los dominios del backend: `auth`, `onboarding`, `subscriptions`, `tenants`, `plans`, `products`, `clients`, `documents`, `sequences`, más `design-system`, `marketing`, `navigation`). Infraestructura transversal en `frontend/lib/` (cliente HTTP, hooks de paginación/fetch, utilidades). Detalle en `modules/frontend.adoc`.

Ver `modules/evidencia.adoc` para patrones transversales con cita de archivo:línea, y `modules/arquitectura_y_flujos.adoc` para el detalle narrativo de los flujos de la sección 5.

4. Interfaces y contratos

El backend expone HTTP vía API Gateway v2 (HTTP API) con autorizador JWT nativo contra el User Pool de Cognito (`infra/stacks/api_stack.py:662-666`). Cada Lambda de dominio enruta internamente por `method+path` (o expresiones regulares para rutas con parámetros) dentro de su propio `handler.py`; el routing inspeccionado se implementa en los handlers y en el registro CDK. Rutas públicas (sin JWT) están señaladas explícitamente en los propios comentarios del código: `/auth/login|refresh|logout|challenge|reset-password`, `/auth/forgot-password` (con `throttling` propio), `/onboarding/otp/request|confirm`, `/subscriptions/payments*` (excepto `/refund`, que exige JWT + rol `superadmin`), `/subscriptions/webhooks/dlocal` (autenticado por HMAC-SHA256, no JWT), y `GET /plans`, `GET /plans/{id}` (pricing público).

Los mensajes asíncronos (SQS) no tienen "ruta": el propio `handler.py` de cada worker rutea por el campo `type` del cuerpo del mensaje (ver `invoice_processor/handler.py:42-62` para el patrón

SIGN/POLL) o procesa un único tipo de evento por cola.

Inventario representativo de endpoints HTTP — ver `matrices/interfaces.csv` (muestra representativa citando archivo:línea real, no exhaustiva dado el volumen: 22 `handler.py` distintos y numerosas rutas HTTP registradas en `api_stack.py`).

Contrato de respuesta HTTP uniforme: envelope `ApiResponse` (`backend/lambda/_base/response.py`) consumido en el frontend por `frontend/lib/api/client.ts:65-82` (`envelope.success/envelope.data/envelope.error`).

5. Flujos técnicos

5.1 Emisión de un comprobante electrónico (factura) hasta autorización SRI

`POST /documents` (JWT, roles `owner|admin|superadmin`) → `documents/handler.py:135-149` (`_emit`, decorado con `@lambda_handler`, `@require_role`, `@idempotent`) → si `doc_type != "04"`, `_emit_invoice` (`documents/handler.py:152-215`) resuelve `tenant` y `plan`, arma las líneas y ejecuta `EmitDocumentUseCase` → `_finalize_emission` (`documents/handler.py:276-310`) persiste el documento en estado `PENDING` (transaccional con la clave de idempotencia) y encola un mensaje `{"type": "SIGN", ...}` en `SIGN_QUEUE_URL` (best-effort: un fallo del `send_message` deja el documento `PENDING` sin bloquear la respuesta `202`, ver comentario en `documents/handler.py:302-308`, riesgo registrado en sección 10).

El worker `invoice_processor` (dos funciones Lambda, `invoice-processor-sign` e `invoice-processor-poll`, mismo código, cada una suscrita a su propia cola — `infra/stacks/api_stack.py:846-913`) recibe el mensaje `SIGN` en `invoice_processor/handler.py:42-51` → `SignDocumentUseCase.execute` (`invoice_processor/use_cases/sign_document.py:39-110`): recupera el documento (idempotente si ya no está `PENDING`, línea 41-47), carga el certificado `.p12` del `tenant` desde `Secrets Manager` (`load_certificate`, línea 50), construye el XML (`xml_builder.build_invoice_xml/build_credit_note_xml`) y lo firma XAdES-BES (`signing.sign_xades_bes`, línea 57), lo envía al SRI vía SOAP (`sri_client.recepcion`, línea 59). Si el SRI lo recibe, pasa el documento a `PROCESSING` y encola un mensaje `POLL` con `delay_seconds=30` (línea 62-74). Si el SRI rechaza permanentemente (clasificado por `sri_error_classifier`), el documento pasa a `REJECTED` y se publica un evento `DocumentRejectedEvent` (líneas 81-101); si el error es reintentable, pasa a `FAILED` con `increment_retry=True` y se relanza la excepción para que SQS reintente (líneas 103-110, `max_receive_count=5` antes de la DLQ de `invoice-sign`, ver `matrices/persistence.csv`).

El mensaje `POLL` es atendido por la misma Lambda (`invoice_processor/handler.py:53-59`) → `PollDocumentUseCase` consulta la autorización SRI y, si es positiva, genera el RIDE (`ride_builder.py`) y sube XML+RIDE a S3 con `Legal Hold` activado (`invoice_processor_poll_fn`, `infra/stacks/api_stack.py:899-905`, `s3:PutObjectLegalHold`).

5.2 Confirmación de pago de suscripción (dLocal Go)

`POST /subscriptions/payments` (público, con `throttling` propio) → `_create_payment` (`subscriptions/handler.py:56-81`, decorado `@idempotent`) crea la orden en dLocal y persiste el `Payment`

en estado inicial. El frontend confirma con `POST /subscriptions/payments/{order_id}/confirm` (público) → `_confirm_payment` (`subscriptions/handler.py:84-112`) → `ConfirmPaymentUseCase.execute` (`subscriptions/use_cases/confirm_payment.py:41-85`): valida el `checkout_token` recibido por header `x-checkout-token` contra el guardado (`hmac.compare_digest`, línea 44-48 — comparación de tiempo constante), rechaza si el pago ya está en un estado no confirmable (`PAID/AUTHORIZED/PENDING`, línea 50-51), llama a `dLocal` (`confirm_payment`) y, según el `status` devuelto, transiciona el pago a `confirm()`, `mark_pending()` o `fail()` (líneas 70-77). En paralelo, `dLocal` puede notificar por `POST /subscriptions/webhooks/dlocal` (público, autenticado por firma HMAC-SHA256 sobre el cuerpo crudo — `_verify_dlocal_signature`, `subscriptions/handler.py:123-125`); el handler solo valida la firma y encola el evento crudo en `DLOCAL_WEBHOOK_QUEUE_URL` (línea 180) para que `workers/dlocal_webhook_processor` lo procese de forma asíncrona y desacoplada de la latencia del webhook.

5.3 Onboarding self-service con OTP

`POST /onboarding/otp/request` (público, throttled 1 req/s ráfaga 5 — `infra/config/dev.yaml`) → `_request_otp` (`onboarding/handler.py:63-99`, `@idempotent`) → `RequestOnboardingOtpUseCase` genera y envía el código; el commit de la verificación es transaccional con la respuesta HTTP vía `require_current_context()/commit_request` (patrón outbox, líneas 93-98). `POST /onboarding/otp/confirm` (`onboarding/handler.py:102-149`) valida el OTP y, según el resultado (`result.tenant` vs `result.lead`), bifurca entre crear el tenant (`_commit_repo().commit_tenant_registration`) o registrar un lead empresarial para contacto manual (`commit_enterprise_lead`) — ambos casos devuelven `201` y comparten el mismo mecanismo de commit transaccional atado a la clave de idempotencia.

Ver `modules/arquitectura_y_flujos.adoc` para el detalle ampliado de estos tres flujos y de dos flujos adicionales (renovación de suscripción por worker programado, y ciclo de vida del certificado digital).

6. Seguridad y contexto de ejecución

Autenticación: Amazon Cognito, un único User Pool para toda la plataforma (`infra/stacks/auth_stack.py:38-73`). `self_sign_up_enabled=False` — todos los usuarios requieren creación administrativa; el comentario del stack describe los roles, y el worker de onboarding crea el owner inicial. El comentario no demuestra un flujo operativo para cada rol. Flujo de login exclusivamente `USER_SRP_AUTH` (`user_password=False` en el cliente web, línea 78-83, comentario explícito "más seguro" para el intercambio SRP con Cognito; esta configuración no describe por sí sola cómo el frontend entrega la contraseña al backend) implementado en `backend/lambda/auth/infra/cognito_auth_provider.py` con SRP real (`infra/srp.py`, uso de `initiate_auth/respond_to_auth_challenge` con `ChallengeName=PASSWORD_VERIFIER`, líneas 40-71). Los parámetros secretos del challenge (`SALT`, `SRP_B`, `SECRET_BLOCK`) se filtran antes de responder al cliente (`_sanitize_challenge_parameters`, `cognito_auth_provider.py:172-183`).

Autorización: `claims custom` en el ID Token — `custom:tenant_id`, `custom:role` (`superadmin|owner|admin|viewer`), `custom:is_superadmin` (`auth_stack.py:9-16`). El API Gateway valida firma/expiración vía `HttpJwtAuthorizer` nativo (`api_stack.py:662-666`); la autorización de rol fino se aplica en los handlers que consumen esos controles con los decoradores `require_superadmin`

`/require_role` de `backend/lambda/_base/permissions.py:29-57` (superadmin siempre pasa; el resto exige pertenecer a la lista de roles permitidos).

Certificados digitales SRI: `backend/shared/certificates/validator.py` — carga el `.p12` con `cryptography.hazmat.primitives.serialization.pkcs12`, valida tamaño máximo `MAX_CERTIFICATE_BYTES = 50 * 1024` (línea 19), exige contraseña no vacía (línea 51-52) y contrasta el RUC del certificado contra el RUC esperado del tenant (`CertificateRucMismatchError`). El endpoint de carga (`certificates/handler.py`) exige `owner/admin` del mismo tenant o superadmin (`_authorize_tenant`, líneas 47-51) y usa `IDEMPOTENCY_TABLE` + reintentos acotados (`_MAX_COMMIT_ATTEMPTS = 3`, línea 32) ante condiciones de carrera al actualizar el mismo tenant. El certificado se almacena en Secrets Manager bajo el prefijo `/codelabs-billing/{env}/tenant/*`, con permisos de Lambda acotados exactamente a ese prefijo (`_grant_certificate_secrets`, `api_stack.py:88-108`). Se confirma la existencia de `backend/lambda/auth/infra/cognito_auth_provider.py` en el commit de `develop` verificado (contenido citado arriba), tal como referencia `AGENTS.md`.

Webhooks externos: el webhook de dLocal Go se autentica por HMAC-SHA256 sobre el cuerpo crudo con comparación de tiempo constante (`_verify_dlocal_signature`, `subscriptions/handler.py:123-125`, `hmac.compare_digest`), no por JWT.

Multitenencia: `BaseRepository` exige `tenant_id` no vacío en el constructor (`shared/db/base_repository.py:51-52`) y construye `PK = "TENANT#{tenant_id}" (:59-60)`. Este control corresponde a los repositorios que usan esa base; las tablas globales, como `tenants`, `plans` y `payments`, tienen claves diferentes y requieren verificar la autorización de sus consumidores.

No se encontraron mecanismos de rate limiting a nivel de aplicación además del throttling de API Gateway por ruta (`api_stack.py:1158-1203`, ver `infra/config/dev.yaml` para los valores configurados). No se encontró un WAF gestionado en los stacks de infra revisados.

7. Persistencia

Persistencia estructurada: **Amazon DynamoDB**, `PAY_PER_REQUEST`, trece tablas definidas en `infra/stacks/database_stack.py` (una por dominio funcional según el comentario del propio archivo, líneas 1-13): `tenants` (GSI `ruc-index`), `audit` (retención 7 años — "retención legal SRI Ecuador", líneas 61), `idempotency` (TTL 24h), `outbox` (DynamoDB Streams `NEW_IMAGE` → relay a SQS), `password-resets`, `plans` (GSI `slug-index`), `clients` (GSI `identification-index`), `products` (GSI `sku-index`), `migrations`, `payments` (GSI `tenant-payments-index`), `sequences` (sin GSI — solo acceso por `pk/sk`), `documents` (GSI `tenant-docs-index`), `batch-jobs` (tabla creada pero declarada en el propio comentario como para una Lambda de un sprint posterior aún no implementada — no se encontró un handler `batch_jobs` en este commit).

Patrón de acceso: `shared/db/base_repository.py` es la base común. Convención de claves `PK = "TENANT#{tenant_id}" / SK = "{PREFIX}#{entity_id}"` (líneas 13-18). Provee: aislamiento multitenant obligatorio (línea 51-52), **soft delete** — nunca se llama `delete_item` directamente, se filtra `Attr("deleted").eq(False)` como `FilterExpression` (líneas 100-131, con una limitación documentada explícitamente en el propio docstring: "DynamoDB applies Limit BEFORE FilterExpression... For lists with many soft-deleted rows, use a GSI with `deleted` as the partition key in later phases", líneas 20-25 — deuda técnica registrada en sección 10), **locking optimista** vía `ConditionExpression/version`

(mapea `ConditionalCheckFailedException` → `OptimisticLockError`, líneas 93-98), **auditoría automática no bloqueante** en cada mutación (`_audit`, líneas 183-206, un fallo del log de auditoría solo genera un warning, no interrumpe la operación de negocio) y helper de `transact_write_items` con mapeo de errores de cancelación transaccional (líneas 157-179).

Almacenamiento de archivos: Amazon S3 (`storage_stack.py`) para XML y RIDE de comprobantes, con **Object Lock (WORM)** y `LegalHold=ON` al escribir el RIDE tras autorización SRI (`invoice_processor/infra/s3_document_storage.py`; `storage_stack.py:46-50`). Los permisos IAM de `api_stack.py:902-905` habilitan Legal Hold; su declaración por sí sola no prueba una escritura.

Secretos: AWS Secrets Manager para credenciales de certificados `.p12` por tenant, credenciales de dLocal Go y API key de Brevo (proveedor de email) — los consumidores citados reciben el **nombre** del secreto como variable de entorno, p.ej. `DLOCALGO_CREDENTIALS_NAME`. Esto describe esas integraciones y no demuestra la ausencia universal de secretos en otros medios.

8. Catálogo de implementación

Conteos verificados sobre el worktree de `develop` (no exhaustivo línea por línea, pero completo a nivel de archivo):

- 22 archivos `handler.py`: 10 dominios HTTP + `invoice_processor` (SQS) + `_base` (decoradores compartidos) + 10 workers (`certificate_expiry_notifier`, `dlocal_webhook_processor`, `email_notifications`, `migrations`, `orphan_payment_notifier`, `outbox_relay`, `pending_activation_reconciler`, `stale_payment_cleaner`, `subscription_renewal_notifier`, `tenant_onboarding`).
- 70 archivos bajo `use_cases/` (excluyendo `__init__.py`) en todo `backend/lambda/`.
- Las entidades usan nombres diversos: `documents/domain/entities.py`, `sequences/domain/entities.py`, `subscriptions/domain/entities/payment.py`, `tenants/domain/tenant.py` y `plans/domain/plan.py`, entre otros. Contar solamente archivos llamados `entities.py` no representa el inventario de entidades.
- 441 archivos `.py` en `backend/` en total; 77 archivos de test unitario (`backend/tests/unit/lambda/<dominio>/` + `backend/tests/unit/shared/`) distribuidos entre los 10 dominios HTTP, `invoice_processor`, `_base` y workers.
- Frontend: 340 archivos `.ts/.tsx`; 43 archivos bajo `features/**/*.screens/`; 61 componentes bajo `features/**/*.components/` + 34 bajo `frontend/components/`; 44 archivos `*.test.ts(x)` (Vitest).
- Infraestructura: 7 stacks CDK (`infra/stacks/*.py`) + `app.py` orquestador + 9 archivos `.py` en total bajo `infra/`.

Ver `matrices/implementation.csv` para una muestra representativa citando archivo:línea (no pretende ser el listado completo de los 70 archivos de casos de uso).

9. Operación y configuración

Comandos declarados en `Makefile` (raíz del repositorio, verificado en el worktree; **no se ejecutó ningún build/test/deploy real como parte de esta documentación**, solo se leyó el archivo):

- `make ci-backend` → `ruff format --check backend/ + ruff check backend/ + coverage run --source=lambdas,shared -m unittest discover -s backend/tests -p "test_*.py" + coverage report`. Prerrequisito declarado: `backend/.venv` con `requirements.txt` + `requirements-ci.txt`.
- `make ci-frontend` → `npm run format:check && npm run lint && npm run typecheck && npm run test:run` (dentro de `frontend/`).
- `make ci-security` → `bandit -r backend/lambdas/ backend/shared/ -c backend/pyproject.toml --severity-level medium + pip-audit -r backend/requirements.txt --strict`.
- `make ci-cdk` → `cdk ls -c env=dev` (valida que los stacks compilen).
- `make ci` → ejecuta los cuatro targets locales. No es un espejo exacto del workflow: este también configura variables AWS, genera XML de cobertura, publica reportes y valida el build web del frontend; el `Makefile` de este commit no incluye ese build.
- `make deploy` → `cdk deploy --all --profile codelabs --require-approval never` (perfil AWS `codelabs`, región `sa-east-1`).
- `make superadmin` / `make token` → scripts operativos (`scripts/create_superadmin.py`, `scripts/make_token.py`).

Configuración por entorno: `infra/config/{dev,staging,prod}.yaml` — claves verificadas (solo nombres, nunca valores sensibles): `env`, `region`, `account` (resuelto vía `CDK_DEFAULT_ACCOUNT`), `superadmin_email`, `dynamodb.billing_mode/point_in_time_recovery/removal_policy`, `cognito.mfa`, `api.throttling`. (por ruta), `domain.hosted_zone/api/frontend`, `cors.origins/cors.headers`. Nombres de secretos referenciados (nunca su valor): `codelabs-billing-{env}/dlocalgo-credentials`, `codelabs-billing-{env}/brevo-api-key`, `/codelabs-billing/{env}/tenant/` (certificados). Configuración del frontend: `frontend/constants/config.ts` resuelve `apiUrl` en cascada (`EXPO_PUBLIC_API_URL` → `extra.apiUrl` de Expo Constants → default por entorno) y expone `dlocalgo.smartFieldsKey/sdkUrl` de forma análoga.

Despliegue: según `app.py` (`infra/app.py:1-24`), "Solo dev por ahora. Prod no se toca hasta tener clientes" — información operativa real del propio código, no inferida. Pipelines CI/CD declarados en `.github/workflows/` (`pr-checks.yml`, `deploy-dev.yml`, `deploy-staging.yml`, `deploy-prod.yml`) — **no se modificó ningún workflow, secreto ni configuración de despliegue** como parte de esta tarea documental; solo se leyeron para describir el estado real.

10. Riesgos y deuda

Clasificación	Hallazgo	Archivo	Línea
Seguimiento recomendado — deuda documentada	El filtro de soft-delete (<code>deleted = false</code>) se aplica como <code>FilterExpression</code> de DynamoDB, que se evalúa después de <code>Limit</code> — el propio docstring señala que esto "mitiga pero no elimina" el problema de páginas con muchos elementos borrados, y propone un GSI futuro con <code>deleted</code> como partition key.	<code>backend/shared/db/base_repository.py</code>	20-25
Seguimiento recomendado — restricción declarada	<code>invoice_processor</code> (SIGN/POLL) no declara <code>reserved_concurrent_executions</code> . Los comentarios atribuyen esta decisión a una cuota de cuenta de 10 ejecuciones y a fallos previos de despliegue. Se verificó la declaración del código; la cuota vigente y el estado de AWS no se consultaron.	<code>infra/stacks/api_stack.py</code>	829-845
Seguimiento recomendado — riesgo funcional	El envío del mensaje SQS <code>SIGN</code> tras persistir un documento en <code>PENDING</code> es "best-effort": si <code>send_message</code> falla, el documento queda en <code>PENDING</code> sin reintento automático (el propio código deja un <code>_log.warning</code> con el texto "document stuck PENDING"). Mismo patrón en <code>_retry</code> y en la marca de anulación por nota de crédito.	<code>backend/lambda/docs/documents/handler.py</code>	302-308, 259-271, 513-519

Clasificación	Hallazgo	Archivo	Línea
Seguimiento recomendado — nomenclatura declarada	El nombre del prefijo de infraestructura (<code>CodeLabsBilling-*</code>) quedó fijo por los recursos ya desplegados; renombrarlo "implicaría recrear recursos" — divergencia consciente entre el nombre de marca actual ("Wali") y el nombre técnico de los stacks CDK.	<code>infra/app.py</code>	1-3
Riesgo operativo declarado	El checkout contiene trabajo de aplicación sin commitear que esta documentación excluye. La nueva comprobación del remoto configurado falla por resolución SSH. La fuente sigue siendo el commit local indicado en la sección 1; regenerar desde la rama remota confirmada cuando se restaure el acceso. Esta es evidencia de instalación, no un hallazgo del código documentado.	<code>AGENTS.md</code>	—

Anexo — Evidencia transversal

Fuente para todo este anexo: worktree aislado creado con `git worktree add --detach <tmp> develop` sobre el commit `09a4a3b2aa87f139c4836733180edf87371c14b3`. Ninguna cita proviene del checkout local con cambios sin commitear.

Patrón de decorador Lambda (contrato compartido de todos los handlers HTTP)

`backend/lambda/_base/handler.py:34-71` define `_build_handler`, con dos fachadas públicas:

`lambda_handler` (`require_tenant=True`, línea 65-66) y `public_lambda_handler` (`require_tenant=False`, línea 69-71). Responsabilidades verificadas en el propio docstring (líneas 6-13) y en el cuerpo de `wrapper` (líneas 36-59):

1. `clear_invocation_context()` al inicio — evita fuga de contexto entre invocaciones en contenedores Lambda reutilizados (línea 37).
2. `Request.from_event(event, require_tenant=...)` parsea el evento API Gateway (línea 41).
3. `bind_invocation_context(request_id, tenant_id, user_id, lambda_name)` propaga contexto a todos los loggers vía `contextvars` (línea 42-47).
4. Captura `AppError` → `ApiResponse.error(exc, request_id)` con el código de la excepción (línea 53-55).
5. Captura `Exception` genérica → loguea el stacktrace completo con `exc_info=True` pero **nunca** lo devuelve al cliente HTTP, solo `InternalError()` (línea 57-59, comentario explícito en el docstring: "the stacktrace NEVER travels to the HTTP response, only to CloudWatch").

El equivalente para colas SQS es `backend/lambda/_base/sqs_handler.py` (`sqs_handler`, usado en `invoice_processor/handler.py:42` y en los 10 workers).

Orden de decoradores en endpoints mutables

Los endpoints de escritura combinan tres decoradores en un orden fijo (de abajo hacia arriba en la aplicación real, de arriba hacia abajo en el código): `@lambda_handler` (o `@public_lambda_handler`) → `@require_role(...)` (cuando aplica) → `@idempotent`. Ejemplo verificado en `backend/lambda/documents/handler.py:135-138`:

```
@lambda_handler
@require_role("owner", "admin", "superadmin")
@idempotent
def _emit(request: Request, context) -> dict:
```

El mismo orden se repite en `certificates/handler.py`, `onboarding/handler.py:63-65` (con `@public_lambda_handler` en vez de `@lambda_handler`, porque el onboarding es anónimo) y `subscriptions/handler.py:56-58`.

Idempotencia HTTP (X-Idempotency-Key)

`backend/lambda/_base/idempotency.py` implementa idempotencia real contra DynamoDB, no solo deduplicación en memoria: la clave de partición combina alcance + clave del cliente (`_scope_for`, línea 82-85: `TENANT#{tenant_id}` o `GLOBAL#{user_id|anonymous}`), y el registro se ata a `method + path + body_hash` (`_matches`, línea 99-104) — una reutilización de la misma clave con un cuerpo distinto lanza `IdempotencyKeyReusedError` (línea 144-145, 257-258) en vez de servir una respuesta cacheada incorrecta. La reserva usa una `ConditionExpression` que permite reintentar si el registro previo expiró por TTL o terminó en `FAILED` (línea 135-139). TTLs distintos por estado: `IN_PROGRESS` 900s, `COMPLETED` 24h, `FAILED` 60s (líneas 48-50). `completion_transact_item()` (línea 209-242) permite que el propio repositorio de dominio cierre la idempotencia **dentro de la misma transacción DynamoDB**

que persiste el cambio de negocio (patrón usado por `onboarding` y `documents`, ver sección 5 del documento principal), en vez de un segundo `update_item` no atómico.

Outbox pattern (eventos de dominio)

`backend/shared/domain/events/outbox.py` + `backend/shared/domain/events/publisher.py` (clase `EventPublisher`, `publisher.py:72`) implementan un outbox transaccional: los eventos se escriben en la tabla `outbox` (DynamoDB Streams `NEW_IMAGE`) dentro de la misma transacción que el cambio de negocio, y `workers/outbox_relay` (suscrito al stream, `infra/stacks/api_stack.py:309-322`) los reenvía a las colas SQS correspondientes (`EVENTS_QUEUE_URL`, `EMAIL_NOTIFICATIONS_QUEUE_URL`) de forma desacoplada. `DirectPublishReason` (`publisher.py:30`) documenta explícitamente los casos donde se permite publicar **sin** pasar por el outbox (p.ej. notificaciones no transaccionales como el email de recuperación de contraseña en `auth/handler.py:50-53`, marcado `DirectPublishReason.NON_TRANSACTIONAL_NOTIFICATION`).

Cliente HTTP del frontend — refresco de token y envelope uniforme

`frontend/lib/api/client.ts:21-83` (`request<T>`) centraliza: adjunta `Authorization: Bearer <idToken>` si `auth=true` (línea 44), revisa expiración local antes de llamar (`isTokenExpired`, línea 36) y dispara `onRefresh()` proactivamente; además reintenta una vez ante un `401` real del servidor (línea 58-63, `isRetry` evita loops infinitos). Toda respuesta pasa por un envelope uniforme `{success, data, error}` (líneas 67-80) validado con un `ZodType<T>` provisto por el llamador — el contrato de tipos vive en `frontend/lib/api/types.ts`.

Sesión y almacenamiento de tokens (frontend)

`frontend/features/auth/store.ts` (Zustand) construye el usuario en memoria decodificando el ID Token (`buildUser`, líneas 21-30) — lee exactamente los mismos claims custom emitidos por Cognito y descritos en `infra/stacks/auth_stack.py` (`custom:role`, `custom:tenant_id`, `custom:is_superuser`). El almacenamiento físico (`frontend/features/auth/storage.ts:11-40`) difiere por plataforma: en nativo usa `expo-secure-store` (keychain/keystore cifrado); en web usa `sessionStorage/localStorage` del navegador según si el token debe sobrevivir el cierre de pestaña (parámetro `persistent`, línea 19-30) — es decir, en web los tokens **no** están en un almacenamiento cifrado del sistema operativo, a diferencia de nativo (dato verificado, no juicio de valor adicional).

Convención de nombres de recursos AWS vs. marca del producto

Todos los recursos CDK usan el prefijo `codelabs-billing-{env}-` (tablas, colas, funciones) y los propios Stacks de nivel superior usan `CodeLabsBilling-{Env}-` (`infra/app.py:52-58`), mientras que el producto se comunica como "Wali" (`frontend/package.json:2` — `"name": "wali-billing"`; dominios `wali-dev.codelabsecuador.com`). Es una divergencia de nomenclatura documentada explícitamente en el propio `infra/app.py:1-3` como decisión consciente para no recrear recursos ya desplegados.

Anexo — Arquitectura y flujos técnicos (detalle ampliado)

Todas las citas de archivo:línea se verificaron sobre el worktree de `develop` (commit `09a4a3b2aa87f139c4836733180edf87371c14b3`).

Flujo A — Emisión de factura hasta autorización SRI (ampliación de la sección 5.1)

```
Frontend (features/documents)
→ POST /documents (JWT, role owner|admin|superadmin)
→ documents/handler.py:135 _emit()
    ├── doc_type == "04" → _emit_credit_note (línea 218)
    └── else → _emit_invoice (línea 152)
        ├── _get_tenant / _get_plan (líneas 155-156)
        ├── EmitDocumentUseCase.execute() (líneas 182-208)
        └── _finalize_emission (línea 210, cuerpo en línea 276)
            ├── ApiResponse.accepted(202, status=PENDING)
            ├── repo.save(document, idempotency=ctx, response) [transaccional]
            └── _send_sign_message() → SQS SIGN_QUEUE_URL [best-effort]
                |
                □

invoice_processor/handler.py:42 handler() [SQS trigger, invoice-processor-sign]
    └── message_type == "SIGN" → SignDocumentUseCase.execute() (sign_document.py:39)
        ├── get(document) – ignora si ya no está PENDING (idempotencia ante reentrega,
        líneas 41-47)
        ├── load_certificate(tenant.certificate_secret_arn) (línea 50, Secrets
        Manager)
        ├── xml_builder.build_invoice_xml / build_credit_note_xml
        ├── signing.sign_xades_bes(xml, private_key, certificate) (línea 57)
        └── sri_client.recepcion(environment, xmls=[signed_xml]) (línea 59,
        SOAP)
            └── según resultado:
                ├── received=True → status=PROCESSING + enqueue_poll(delay=30s) (líneas
                61-74)
                ├── error PERMANENT → status=REJECTED + DocumentRejectedEvent (líneas
                81-101)
                └── error reintentable → status=FAILED, increment_retry, raise (líneas
                103-110)
                    |
                    □

invoice_processor/handler.py:53 message_type == "POLL"
    └── PollDocumentUseCase.execute()
        ├── sri_client – consulta autorización (AutorizacionComprobantesOffline)
        └── si autorizado: ride_builder → S3 (XML + RIDE, LegalHold=ON,
        api_stack.py:899-905)
```

Colas involucradas (`infra/stacks/queues_stack.py`, ver `matrices/persistence.csv` para DLQ): `invoice-sign` (visibility 720s, `max_receive_count=5`), `invoice-poll` (idéntico), ambas consumidas por las mismas dos funciones Lambda `invoice-processor-sign/invoice-processor-poll` (mismo código, distinta cola de origen — `api_stack.py:846-913`).

Riesgo funcional real (no hipotético) en este flujo: el `send_message` de `_send_sign_message` (`documents/handler.py:123-129`) se ejecuta **después** de confirmar la persistencia del documento y está envuelto en un `try/except Exception` que solo loguea un `warning` (líneas 302-308) — si SQS no está disponible en ese instante, el documento queda `PENDING` indefinidamente sin un mecanismo de reconciliación automática visible en el código inspeccionado (a diferencia de los pagos, que sí tienen `workers/pending_activation_reconciler` y `workers/stale_payment_cleaner`). Registrado en la sección 10 del documento principal.

Flujo B — Pago de suscripción con dLocal Go SmartFields (ampliación de la sección 5.2)

```
Frontend (features/subscriptions/use-3ds-flow.ts, use-dlocal-smartfields.ts)
  → POST /subscriptions/payments (público, throttled)
  → subscriptions/handler.py:56 _create_payment()
    └─ CreatePaymentUseCase → dLocal crea orden → Payment persistido

  → POST /subscriptions/payments/{order_id}/confirm (público)
  → subscriptions/handler.py:84 _confirm_payment()
    └─ ConfirmPaymentUseCase.execute() (confirm_payment.py:41-85)
      └─ hmac.compare_digest(payment.checkout_token, header) (línea 44-48)
      └─ rechaza si status ∉ {PAID, AUTHORIZED, PENDING} (línea 50-51)
      └─ dlocal.confirm_payment(...) (línea 53-62)
      └─ transición: confirm() | mark_pending() | fail() (líneas 70-77)

  □ (asíncrono, iniciado por dLocal)
  → POST /subscriptions/webhooks/dlocal (público, HMAC-verificado)
  → subscriptions/handler.py:171 _dlocal_webhook()
    └─ _verify_dlocal_signature(raw_body, x-signature, secret_key) (línea 123-125)
    └─ _enqueue_dlocal_webhook() → SQS dlocal-webhook-queue (línea 128-144)

workers/dlocal_webhook_processor/handler.py [SQS trigger]
  └─ procesa el evento de forma desacoplada de la latencia del webhook HTTP
```

Renovación automática (no disparada por el usuario): `workers/subscription_renewal_notifier` corre diario (`cron(0 10 * * ? *)`, `api_stack.py:518-522`) → `NotifySubscriptionRenewalUseCase.execute()` (`workers/subscription_renewal_notifier/use_case.py:85-136`) recorre tenants con vencimiento próximo (`list_with_subscription_expiry_due`) y por cada uno: envía recordatorio a 7 días (`_REMINDER_DAYS`, línea 24), o si ya venció, intenta auto-cobro con el `dlocal_payer_id` guardado

(`_try_auto_charge`, línea 196) usando un **marcador de reconciliación** (`create_auto_renewal_reconciliation_marker`, línea 217) para evitar doble cobro si el proceso se reintenta a medio camino — si el marcador ya existe, no cobra de nuevo y alerta para reconciliación manual (`auto_charge_reconciliation_required`, líneas 225-247). Si `dLocal` falla o el guardado transaccional posterior al cobro exitoso falla, también se marca para reconciliación manual en vez de perder el cobro silenciosamente (`_AutoRenewalReconciliationRequiredError`, líneas 257-320). Tenants en `PAYMENT_FAILED` tienen una gracia de 7 días (`_PAYMENT_FAILED_GRACE_DAYS`, línea 25) antes de expirar la suscripción definitivamente.

Flujo C — Onboarding self-service con OTP (ampliación de la sección 5.3)

```
POST /onboarding/otp/request (público, throttle burst=5 rate=1)
  → onboarding/handler.py:63 _request_otp()
    └─ RequestOnboardingOtpUseCase.execute() → genera OTP, decide self_service
vs lead
    └─ _verification_repo().commit_request(verification, events, idempotency,
response)

POST /onboarding/otp/confirm (público, throttle burst=10 rate=2)
  → onboarding/handler.py:102 _confirm_otp()
    └─ ConfirmOnboardingOtpUseCase.execute()
    └─ bifurca por resultado:
      └─ result.tenant → _commit_repo().commit_tenant_registration(...) (201,
tenant creado)
      └─ result.lead → _commit_repo().commit_enterprise_lead(...) (201,
"Te contactaremos pronto")
```

Ambas ramas comparten el mismo mecanismo transaccional: la respuesta HTTP ya construida (`ApiResponse.created`) se pasa como argumento al método de commit del repositorio, que la serializa como parte de la misma transacción DynamoDB que cierra la idempotencia (`completion_transact_item`, ver `modules/evidencia.adoc`) — evita el caso donde el negocio se persiste pero la respuesta de idempotencia queda desincronizada.

El worker asociado, `workers/tenant_onboarding`, consume la cola `tenant-onboarding` (`api_stack.py:326-357`) para crear el usuario `owner` en Cognito (`cognito-idp:AdminCreateUser`) de forma asíncrona tras el registro — desacopla la latencia de Cognito de la respuesta HTTP del OTP.

Flujo D — Ciclo de vida del certificado digital (resumen)

`PUT /tenants/{id}/certificate` (JWT; `owner/admin` del mismo tenant o superadmin, `_authorize_tenant`, `certificates/handler.py:47-51`) → `UpdateCertificateUseCase` valida el `.p12` con `CertificateValidator.validate_base64` (tamaño máximo 50KB, RUC del certificado vs. RUC del tenant, contraseña no vacía — `shared/certificates/validator.py:22-42`) y lo guarda en Secrets Manager vía `CertificateStore`; tras el commit se hace **best-effort** bootstrap del punto de emisión de

pruebas 001/099 (`_bootstrap_testing_point`, `certificates/handler.py:56-63`). El worker diario `workers/certificate_expiry_notifier` (`cron(0 9 * * ? *)`) alerta a los tenants 60 y 30 días antes del vencimiento del certificado (comentario en `api_stack.py:430-432`).

Anexo — Backend (**backend/**)

Muestra representativa dado el volumen del código (441 archivos `.py`, 70 archivos de casos de uso, excluyendo `__init__.py`, 22 `handler.py`). El análisis es representativo. Las rutas y roles se contrastan con los handlers citados y su registro en `infra/stacks/api_stack.py`; los comentarios de cabecera por sí solos no demuestran autorización.

Capa compartida (**backend/shared/**)

- `shared/db/base_repository.py` — repositorio base (ver cita completa en sección 7 del documento principal).
- `shared/db/client.py`, `db/transactions.py`, `db/paginator.py` (cursores opacos base64 para paginación), `db/counts.py` (`paginated_count`, usado por `_count_raw`), `db/locks.py` (locks transaccionales de unicidad, p.ej. RUC de tenant, SKU de producto, identificación de cliente — patrón mencionado explícitamente en los comentarios de `database_stack.py`, líneas 39-40, 140-141, 163).
- `shared/audit/writer.py` — construye el ítem de auditoría (`audit_item`) consumido por `BaseRepository._audit` (`base_repository.py:183-206`).
- `shared/certificates/` — `validator.py` (validación .p12, ver sección 6), `store.py` (persistencia en Secrets Manager), `metadata.py`, `expiry.py`, `errors.py`.
- `shared/secrets/client.py` — `get_secret_json`, usado por `auth`, `subscriptions`, `tenants` para leer credenciales de dLocal Go sin exponerlas como variables de entorno con valor literal.
- `shared/domain/events/` — `outbox.py`, `publisher.py` (`EventPublisher`, `DirectPublishReason`), `domain_event.py` (ver `modules/evidencia.adoc`).
- `shared/domain/base_entity.py` — `TenantScopedEntity`, tipo base referenciado por `BaseRepository._to_item/_from_item` (abstractos, `base_repository.py:210-216`).
- `shared/domain/value_objects/` — incluye `ruc.py` (value object RUC, usado en `CertificateValidator.validate_bytes`, `validator.py:54`, para normalizar y comparar el RUC del certificado contra el del tenant).
- `shared/logger.py` — logging estructurado; `bind_invocation_context/clear_invocation_context` consumidos por `_base/handler.py` y `_base/sqs_handler.py`.
- `shared/errors.py` — jerarquía `AppError` (`NotFoundError`, `ForbiddenError`, `ValidationError`, `OptimisticLockError`, `DatabaseError`, `ExternalServiceError`, `IdempotencyInProgressError`, `IdempotencyKeyReusedError`, entre otras) mapeada a códigos HTTP por `ApiResponse.error` (`_base/response.py`).
- `shared/billing.py` — cálculo de precios (`gross_price`, `plan_net_price`), usado por `subscriptions` y por `workers/subscription_renewal_notifier/use_case.py:210`.
- `shared/dates.py` — `isoformat_ecuador`, `now_utc` (normalización horaria a la zona de Ecuador para

fechas visibles al usuario/SRI).

Capa transversal de Lambda (**backend/lambda/_base/**)

`handler.py` (decoradores HTTP), `sqs_handler.py` (decorador para colas), `permissions.py` (`require_superuser`, `require_role`), `idempotency.py` (ver `modules/evidencia.adoc`), `parser.py` (`Request.from_event`, `parse`, `require_path_param`), `response.py` (`ApiResponse` — envelope uniforme `{success, data, error}`), `query_params.py` (`parse_enum_query_param`, `parse_list_limit`).

Dominios HTTP — estructura verificada

Dominio	Subcarpetas presentes	Rutas HTTP representativas y fuente
auth	<code>domain/</code> (<code>commands</code> , <code>errors</code> , <code>events</code> , <code>password_reset_results</code> , <code>repositories/i_auth_provider.py</code>), <code>infra/</code> (<code>cognito_auth_provider.py</code> , <code>password_reset_repository.py</code> , <code>srp.py</code>), <code>use_cases/</code> (6), <code>schemas.py</code>	POST <code>/auth/{login,refresh,logout,challenge,forgot-password,reset-password}</code> — todas públicas (<code>auth/handler.py:6-12</code>).
onboarding	<code>domain/</code> , <code>infra/</code> (<code>enterprise_lead_repository.py</code> , <code>identity_provider.py</code> , <code>onboarding_commit_repository.py</code> , <code>onboarding_verification_repository.py</code> , <code>plan_catalog.py</code>), <code>use_cases/</code> , <code>schemas.py</code>	POST <code>/onboarding/otp/{request,confirm}</code> — públicas, throttled (<code>onboarding/handler.py:160-166</code>).
subscriptions	<code>domain/entities/payment.py</code> , <code>domain/repositories/</code> (<code>i_dlocal_client</code> , <code>i_payment_repository</code> , <code>i_plan_catalog</code>), <code>infra/</code> (<code>dlocal_client.py</code> , <code>payment_repository.py</code> , <code>plan_catalog.py</code>), <code>use_cases/</code> (5: <code>confirm_payment</code> , <code>create_payment</code> , <code>get_payment</code> , <code>process_webhook</code> , <code>refund_payment</code>)	POST <code>/subscriptions/payments</code> , POST <code>.../confirm</code> , GET <code>.../{order_id}</code> , POST <code>.../refund</code> (JWT superadmin), POST <code>/subscriptions/webhooks/dlocal</code> (HMAC) — <code>subscriptions/handler.py:6-11</code> .
tenants	<code>domain/</code> (<code>enums</code> , <code>tenant.py</code> , <code>events</code> , <code>dashboard_summary.py</code> , <code>repositories/</code>), <code>infra/</code> (<code>payment_reader.py</code> , <code>plan_catalog.py</code> , <code>tenant_repository.py</code>), <code>use_cases/</code>	12 casos de uso: CRUD, cambio de plan/status, activación/renovación/reintento de suscripción, reintento de onboarding, dashboard superadmin (<code>tenants/handler.py:5-18</code> , roles mixtos <code>superadmin</code> <code>owner</code> <code>admin</code> según ruta).
certificates	<code>use_cases/</code> (<code>get_certificate</code> , <code>update_certificate</code>), sin <code>domain/</code> / <code>infra/</code> propios — reutiliza <code>shared/certificates/</code> y <code>tenants/infra/tenant_repository.py</code>	GET/PUT <code>/tenants/{id}/certificate</code> (JWT, mismo tenant y rol permitido, o superadmin).

Dominio	Subcarpetas presentes	Rutas HTTP representativas y fuente
sequences	domain/entities.py, infra/sequences_repository.py, use_cases/	GET/POST /tenants/{id}/establishments, POST/PATCH .../emission- points/{point}} (api_stack.py:761-772).
products	domain/, infra/, use_cases/	CRUD /products, GET/PUT /products/discount-campaign (api_stack.py:741-755).
clients	domain/, infra/, use_cases/	CRUD /clients (api_stack.py:723-735).
plans	domain/plan.py, domain/commands.py, domain/errors.py, infra/plan_repository.py, use_cases/ (create_plan, get_plan, list_plans, toggle_plan, update_plan)	GET /plans, GET /plans/{id} públicas (pricing); POST/PATCH /plans*, GET /superadmin/plans* bajo JWT (api_stack.py:997- 1021).
documents	domain/entities.py, domain/commands.py, domain/errors.py, infra/ (discount_campaign_catalog, documents_repository, plan_reader, product_catalog, sequences_adapter), use_cases/ (9: annul_document, emit_credit_note, emit_document, get_document, get_documents_summary, get_ride_url, get_xml_url, list_documents, retry_document)	Ver cita completa en sección 5.1 y documents/handler.py:6-23 (incluye nota de deprecación de annul en favor de nota de crédito, línea 15-16).
invoice_processor	Sin schemas.py (no HTTP); módulos de dominio propios en la raíz: events.py, ports.py, ride_builder.py, signing.py, sri_client.py, sri_error_classifier.py, sri_error_mapper.py, xml_builder.py; infra/ (certificate_loader, s3_document_storage, sqs_event_publisher); use_cases/ (poll_document, sign_document)	Sin rutas HTTP — consumidor SQS puro, rutea por type (SIGN POLL) del mensaje (invoice_processor/handler.py:4 2-62).

Workers (backend/lambda/workers/) — disparadores verificados en infra/stacks/api_stack.py

Worker	Disparador	Propósito (comentario real del código)
outbox_relay	DynamoDB Stream (tabla outbox)	Reenvía eventos del outbox a las colas SQS correspondientes (api_stack.py:292-322).
tenant_onboarding	SQS tenant- onboarding	Crea el usuario owner en Cognito tras el registro (api_stack.py:324-357).

Worker	Disparador	Propósito (comentario real del código)
email_notifications	SQS email-notifications	Envío de correo transaccional vía Brevo (api_stack.py:359-403).
migrations	Invocación manual/CI-CD	Ejecuta migraciones de datos una sola vez, registradas en MIGRATIONS_TABLE (api_stack.py:405-428).
certificate_expiry_notifier	EventBridge cron(0 9 * * ? *)	Alerta 60/30 días antes del vencimiento del certificado .p12 (api_stack.py:430-470).
subscription_renewal_notifier	EventBridge cron(0 10 * * ? *)	Recordatorio de renovación + expiración + auto-cobro con reconciliación (api_stack.py:472-522, detalle en modules/arquitectura_y_flujos.adoc).
orphan_payment_notifier	EventBridge cron(0 * * * ? *) (cada hora)	Detecta pagos PAID sin tenant vinculado y alerta al superadmin (api_stack.py:524-565).
stale_payment_cleaner	EventBridge cron(15 * * * ? *) (cada hora)	Cancela localmente pagos CREATED/PENDING abandonados por 24h (STALE_PAYMENT_GRACE_MINUTES=1440, api_stack.py:567-591).
pending_activation_reconciler	EventBridge rate(5 minutes)	Activa tenants con pago confirmado pero cuya activación local se perdió (api_stack.py:593-621).
dlocal_webhook_processor	SQS dlocal-webhook-queue	Procesa el webhook de dLocal ya verificado por firma, de forma asíncrona (api_stack.py:1099-1121).

Tests (backend/tests/)

77 archivos `test_*.py` bajo `backend/tests/unit/lambda/<dominio>/` (un subdirectorio por cada uno de los 10 dominios HTTP más `invoice_processor`, `_base` y `workers`) y `backend/tests/unit/shared/`. Configuración de cobertura en `backend/pyproject.toml`: `fail_under = 70`, fuente `lambda,shared`, excluye `.venv`, `tests`, `migrations`. No se ejecutó la suite como parte de esta documentación (fuera de alcance: solo lectura del worktree); el conteo de archivos de test es evidencia directa de que **todos** los dominios HTTP tienen al menos un test unitario, no evidencia del resultado de ejecutarlos.

Anexo — Frontend (frontend/)

Muestra representativa dado el volumen del código (340 archivos `.ts/.tsx`, 43 pantallas, 95 componentes entre `features/**/components/` y `frontend/components/`, 44 archivos de test Vitest). No se documenta cada pantalla individualmente; se documenta la estructura de enrutamiento (evidencia completa por ser un árbol de archivos pequeño y determinístico) y los módulos transversales (`lib/`) con cita real.

Enrutamiento (**frontend/app/**, Expo Router — convención de archivos)

Grupos de layout verificados (carpetas entre paréntesis no generan segmento de URL, son agrupadores de layout de Expo Router):

- (**public**)/ — `_layout.tsx`, `components.tsx` (showcase de design system), `legal/{privacy,refund,terms}.tsx`, `register/{details,otp,confirm}.tsx` (flujo de onboarding público, consume `features/onboarding`).
- (**auth**)/ — `_layout.tsx`, `login.tsx`, `challenge.tsx` (reto Cognito, p.ej. `NEW_PASSWORD_REQUIRED`), `forgot-password.tsx`, `reset-password.tsx`.
- (**app**)/ — `_layout.tsx` (guardia de sesión — requiere `hydrated && idToken`, ver `frontend/app/(app)/_layout.tsx:9-16` y el selector `selectIsAuthenticated` de `frontend/features/auth/store.ts`), `confirm-plan.tsx`, `activate-subscription.tsx`, `upload-certificate.tsx` (pasos post-registro compartidos por cualquier rol autenticado).
 - (**app**)/(**tenant**)/ — `_layout.tsx`, `dashboard.tsx`, `billing.tsx`, `clients/`, `products/`, `documents/`, `credit-notes/`, `settings/` — área operativa del tenant (`owner`|`admin`|`viewer`).
 - (**app**)/(**superadmin**)/ — `_layout.tsx`, `dashboard.tsx`, `tenants/`, `plans/` — área exclusiva de plataforma.
- Raíz: `_layout.tsx` (layout global de la app), `index.tsx` (landing / redirección según sesión), `+html.tsx` (shell HTML para el build web de Expo Router).

La separación de carpetas (**tenant**) vs. (**superadmin**) bajo el mismo grupo (**app**) refleja directamente los mismos dos ámbitos de autorización que el backend expone vía `custom:role` / `custom:is_superadmin` (ver sección 6 del documento principal) — no hay un tercer layout para **viewer**: ese rol comparte el layout de (**tenant**) con acceso de solo lectura controlado a nivel de componente/hook, no de ruta (no se encontró una ruta exclusiva de **viewer** en el árbol de **app**).

Features (**frontend/features/<dominio>/**) — paralelismo con dominios backend

Cada carpeta de dominio replica, con variaciones, el patrón `api.ts` (llamadas HTTP tipadas) + `schemas.ts` (validación Zod) + `types.ts` + `hooks/` + `screens/` + `components/`: `auth`, `clients`, `documents`, `onboarding`, `plans`, `products`, `sequences`, `subscriptions`, `tenants`. Dos features sin contraparte 1:1 en el backend: `design-system` (catálogo visual interno, sin API propia) y `marketing/navigation` (landing pública y navegación entre pantallas, respectivamente — capas de presentación puras).

Ejemplo verificado de la profundidad real de una feature (`subscriptions/`): `api.ts`, `schemas.ts`, `dlocal-types.ts`, `dlocal-field-options.ts`, `use-dlocal-smartfields.ts` (integración con el SDK externo de dLocal Go SmartFields cargado dinámicamente, ver `frontend/constants/config.ts:38-42`, `resolveDLocalGoSdkUrl`), `use-3ds-flow.ts` + `use-3ds-flow.test.ts` (manejo del reto 3-D Secure del banco emisor durante la confirmación de pago).

Infraestructura transversal (**frontend/lib/**)

- **lib/api/client.ts** — cliente HTTP central (ver cita completa en **modules/evidencia.adoc**), **errors.ts** (**ApiError**), **types.ts** (contrato **ApiEnvelope**), con tests propios (**client.test.ts**, **idempotency.test.ts**).
- **lib/hooks/** — **useFetch**, **useAsync**, **useFormSubmit**, **useCursorPagedList/useEagerPagedList/useLocalPagedItems/usePaginatedList** (variantes de paginación consumidas por las listas de **clients**, **products**, **documents**, **tenants**), **useDebounceSearch**, **useRefreshOnFocus**, **useNow**, **useCertificateFilePicker**, **useIsDesktopLayout** — 8 de estos 12 hooks tienen test unitario dedicado (***.test.ts** junto al hook).
- **lib/utils/** — **jwt.ts** (**decodeJwtClaims**, **isTokenExpired**, consumidos por **features/auth/store.ts** y **lib/api/client.ts**), **ruc.ts** (validación de RUC ecuatoriano, con test), **date-range.ts**, **ecuador-time.ts**, **form-validators.ts**, **format.ts**, **retry.ts**.
- **lib/theme-context.tsx** — contexto de tema claro/oscuro para el design system.

Autenticación y sesión (**frontend**)

features/auth/store.ts (Zustand, ver cita completa en **modules/evidencia.adoc**) mantiene **user**, **idToken**, **accessToken**, **refreshToken**, **hydrated** en memoria; **features/auth/storage.ts** persiste los tokens según plataforma (**expo-secure-store** en nativo, **sessionStorage/localStorage** en web). **features/auth/api.ts** implementa las llamadas a **/auth/***; **features/auth/schemas.ts** valida los formularios de login/reset con Zod (con test **schemas.test.ts**); **features/auth/hooks/** expone hooks de conveniencia sobre el store.

Configuración de entorno (**frontend/constants/config.ts**)

Resolución en cascada verificada (líneas 9-27): variable de entorno de build **EXPO_PUBLIC_API_URL** → **Constants.expoConfig.extra.apiUrl** (inyectado en CI/CD) → default hardcodeado por entorno (**DEV_API_URL**, **STAGING_API_URL**, **PROD_API_URL**, todas bajo **codeLabEcuador.com**, líneas 5-7) → si **__DEV__** usa el default de dev. Mismo patrón para **EXPO_PUBLIC_DLOCALGO_SMARTFIELDS_KEY** (clave pública, no secreta, del SDK de checkout de dLocal Go).

Tests (**frontend**)

44 archivos **.test.ts/.test.tsx** con Vitest (**npm run test:run** en **make ci-frontend**), distribuidos junto a los archivos que prueban (no en un directorio **__tests__**/ centralizado) — patrón consistente en **lib/hooks/**, **lib/utils/**, y varias **features/<dominio>**. No se detectaron tests end-to-end (Playwright/Detox) en el árbol inspeccionado; la cobertura verificada es unitaria/de integración ligera (hooks, schemas, formularios, cliente HTTP).